

Documentación técnica

Anexo a la memoria individual

Proyecto	ProceduralEcosystem — simulación y generación procedural de un ecosistema forestal
Tecnología	Unreal Engine 5.7 · C++ · módulo ProceduralEcosystem (Runtime)
Repositorio	Izana-L/ProceduralEcosystem
Autor	
Tutor/a	
Fecha	29 de agosto de 2026

Documento redactado en castellano. Los diagramas siguen la notación UML 2 y se han generado con PlantUML a partir de los ficheros fuente incluidos en Docs/AnexoTecnico/diagramas/fuentes/.

Índice

Componentes

- 1.1. Criterio de descomposición y notación empleada
- 1.2. Diagrama de componentes
- 1.3. Descripción de cada componente
- 1.4. Dependencias: lectura del diagrama

Bibliotecas de terceros

- 2.1. Resumen
- 2.2. Módulos del motor utilizados
- 2.3. Plugins del motor
- 2.4. Herramientas externas de apoyo
- 2.5. Recursos artísticos
- 2.6. Por qué no se han usado bibliotecas externas

Clases

- 3.1. Alcance, notación y convenios de representación
- 3.2. Núcleo
- 3.3. Configuración y datos de especie
- 3.4. Terreno, campos de recurso y luz
- 3.5. Simulación
- 3.6. Ecología
- 3.7. Geometría procedural (I): esqueleto
- 3.8. Geometría procedural (II): mallado, follaje y viento
- 3.9. Render (I): gestor de nivel de detalle
- 3.10. Render (II): biblioteca de arquetipos
- 3.11. Capa de suelo, instrumentación y depuración

Requisitos para usar el proyecto

- 4.1. Requisitos de hardware
- 4.2. Software necesario
- 4.3. Instalación paso a paso
- 4.4. Configuración del proyecto
- 4.5. Puesta en marcha y comprobación
- 4.6. Ejecución de las pruebas automáticas
- 4.7. Problemas conocidos y su solución

Estructura del proyecto

- 5.1. Vista general del repositorio
- 5.2. Separación de código propio, terceros, recursos y configuración
- 5.3. Organización del código fuente
- 5.4. Recursos (Content/)
- 5.5. Configuración
- 5.6. Artefactos generados y control de versiones
- 5.7. Observaciones sobre la estructura

1. Componentes

1.1. Criterio de descomposición y notación empleada

Todo el código propio del proyecto se compila en un único módulo de Unreal Engine llamado `ProceduralEcosystem`, de tipo *Runtime*. Un módulo, sin embargo, no es una unidad de diseño: es una unidad de compilación y enlace. Para este documento se ha tomado como **componente** cada agrupación de tipos que (a) cubre una responsabilidad única y bien delimitada, (b) se puede sustituir o reescribir sin tocar el resto y (c) expone al resto del sistema un conjunto reducido y estable de operaciones. Cada componente se corresponde con una carpeta del árbol de código fuente, de modo que la descomposición lógica y la organización física del proyecto coinciden y se pueden verificar la una contra la otra.

1.2. Diagrama de componentes

El diagrama de la figura 1 sigue la notación UML 2 para diagramas de componentes:

- Los rectángulos con el icono de componente representan los componentes del sistema.
- Las **interfaces provistas** se dibujan con la notación de «pirulí» (circunferencia unida al componente que la implementa).
- Las **interfaces requeridas** se dibujan como flechas discontinuas de dependencia («use») desde el componente consumidor hacia la interfaz, formando el conector de ensamblado bola-casquillo característico de UML 2.
- Los componentes sombreados con el estereotipo «*software de terceros*» no forman parte del código propio.

Las interfaces del diagrama no son clases `interface` del lenguaje —C++ y el sistema de reflexión de Unreal no lo requieren para este caso—, sino el **contrato efectivo** que cada componente ofrece: el subconjunto de sus operaciones públicas del que dependen los demás. Se han nombrado por lo que aportan, no por el tipo que las implementa, precisamente para que el diagrama siga siendo válido si mañana cambia la clase que las materializa.

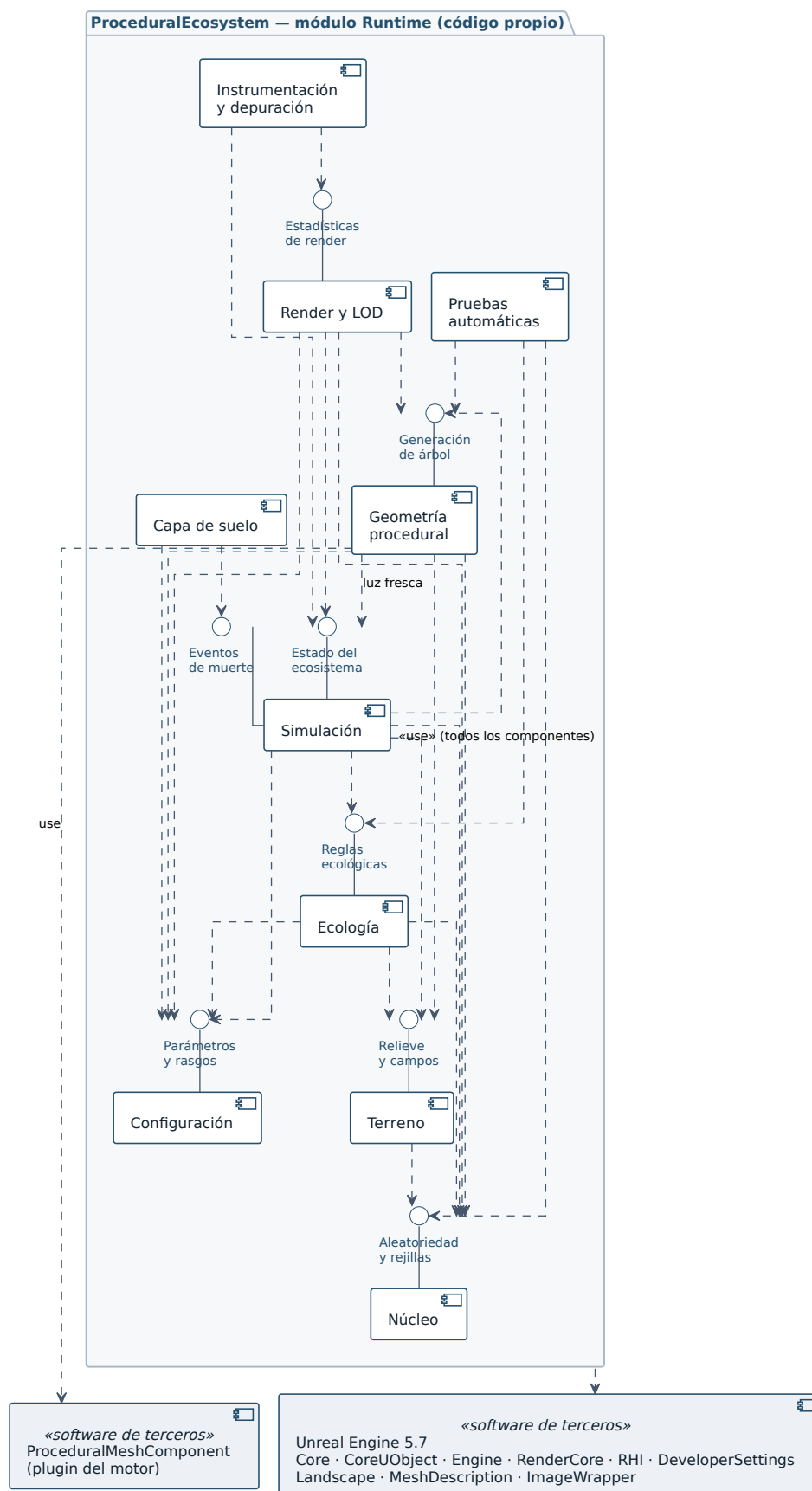


Figura 1. Diagrama de componentes (UML 2). Los componentes consumidores se sitúan en la parte superior y los proveedores debajo, de modo que todas las dependencias del código propio se lean de arriba abajo.

1.3. Descripción de cada componente

Componente	Carpeta	Responsabilidad	Interfaz provista
Núcleo	Core	Utilidades transversales sin estado: generador de números aleatorios determinista (<i>xorshift32</i>) con un flujo independiente por subsistema, aritmética de rejillas (mundo ↔ celda, indexado de vóxeles, índice espacial CSR), ley de extinción de Beer-Lambert, geometría vectorial estable e instrumentación para las herramientas del motor.	Aleatoriedad y rejillas
Configuración	Config Species	Única fuente de parámetros del sistema. <i>UEcosystemSettings</i> expone ciento treinta y cinco ajustes en <i>Project Settings</i> , persistidos en <i>DefaultGame.ini</i> ; <i>USpeciesData</i> describe los rasgos de cada especie como recurso editable.	Parámetros y rasgos
Terreno	Terrain	Generación del relieve (ruido fractal con <i>domain warping</i> y crestas, más erosión hidráulica y térmica), campos escalares 2D de agua (índice topográfico de humedad) y nutrientes, y rejilla volumétrica de luz de escala de paisaje con atenuación exponencial por área foliar acumulada.	Relieve y campos
Ecología	Ecology	Modelo biológico en forma de funciones puras: crecimiento logístico, alometría, acumulación y recuperación de estrés, mortalidad por edad y por condición, senescencia y supresión, producción y dispersión de semillas, germinación, consumo de recursos y curvas de respuesta del vigor. Incluye las estructuras de datos de la población (formato SoA), el <i>hash</i> espacial, los depósitos de recurso con doble búfer y el espacio de trabajo por tarea.	Reglas ecológicas
Simulación	Simulation	Orquestador del sistema: posee el estado del ecosistema y ejecuta el <i>tick</i> con un orden de etapas fijo. Desacopla el paso de simulación del fotograma, aplica el presupuesto de tiempo por fotograma, guarda y carga estados (« <i>bake</i> » a un año objetivo) y publica los datos que consumen las capas de vista.	Estado del ecosistema Eventos de muerte
Geometría procedural	Geometry	Construcción de un árbol individual: nube de atractores dentro de la envolvente de copa, algoritmo de colonización del espacio, rejilla fina de luz para fototropismo y autopoda, radios por modelo de tubería, deformación del tronco, mallado del leño y del follaje y canales de viento por vértice. Incluye el actor <i>AHeroTreeActor</i> , que muestra el resultado con un <i>UProceduralMeshComponent</i> .	Generación de árbol
Render y LOD	Render	Decide cómo se dibuja cada árbol vivo (nivel <i>hero</i> , instancia, <i>impostor</i> o fuera de rango) con histéresis para evitar oscilaciones, hornea mallas estáticas por arquetipo (especie × tramo de tamaño × variante), gestiona los componentes de malla instanciada jerárquica y propaga estación y viento a los materiales mediante <i>Material Parameter Collections</i> .	Estadísticas de render
Capa de suelo	Render	Materializa la necromasa: consume los eventos de muerte de la simulación y coloca tocones —con su ciclo en pie, caída, tronco tumbado y retirada— y hojarasca como instancias.	—

Componente	Carpeta	Responsabilidad	Interfaz provista
Instrumentación y depuración	Debug	Visualizador de campos como textura dinámica para los mapas de calor, perfilador de fotograma con HUD y captura a CSV, auditoría de especies y exportación del mapa de alturas a PNG de 16 bits.	—
Pruebas automáticas	Test	Treinta y una pruebas del marco de automatización del motor sobre las partes deterministas y sin estado global: generador aleatorio, fórmulas ecológicas, relieve, rejillas de luz, geometría del árbol y cálculo de nivel de detalle.	—

1.4. Dependencias: lectura del diagrama

Estratificación. Las dependencias del código propio forman un grafo estratificado con un único ciclo, del que se habla más abajo. *Núcleo* y *Configuración* son la capa base y no dependen de ningún otro componente propio; sobre ellos se apoyan *Terreno* y *Ecología*; sobre estos, *Simulación*; y por encima, las capas de vista (*Geometría procedural*, *Render* y *LOD*, *Capa de suelo*) y de observación (*Instrumentación*, *Pruebas*). Esta estratificación es la que permite probar el modelo biológico sin cargar un mundo y cambiar la representación visual sin tocar la simulación.

La población es la única fuente de verdad. *Render* y *LOD* depende de *Simulación* únicamente en modo de lectura: obtiene la población mediante una referencia constante y elige representación sin modificar un solo dato del modelo. La consecuencia práctica es que desactivar por completo la capa de render no altera la evolución del ecosistema, lo que hace comparables las corridas con y sin visualización.

Acoplamiento por eventos con la capa de suelo. *Capa de suelo* no consulta la población: consume un anillo circular de eventos de muerte mediante un cursor monótono. Al no depender de los índices de la población —que se compactan en cada *tick* al eliminar los individuos muertos—, el componente es inmune a esa reordenación y puede procesar los eventos con el retraso que le convenga.

El único ciclo del grafo, y por qué está controlado. Existe una dependencia mutua entre *Simulación* y *Geometría procedural*: la simulación crea árboles *hero* a través de `SpawnHeroTree()`, y el actor generado necesita el campo de luz gruesa del bosque para que sus ramas crezcan contra la sombra real de sus vecinos. El ciclo se controla de dos maneras. Primero, la dependencia inversa vive exclusivamente en el fichero de implementación (`HeroTreeActor.cpp`) y no en ninguna cabecera pública, de modo que no se propaga por inclusión al resto del sistema. Segundo, el actor **no cachea** un puntero al campo de luz —que es una estructura propiedad del subsistema y no reflejada— sino que lo solicita al subsistema vivo en cada reconstrucción: así se evita que un actor que sobreviva al subsistema desreferencie memoria liberada.

Dependencia con la plataforma. Todos los componentes dependen del motor, por lo que en el diagrama esa relación se representa una sola vez, a nivel de paquete, en lugar de repetir diez flechas idénticas. La dependencia del plugin `ProceduralMeshComponent` sí es específica: solo la necesita *Geometría procedural*, que es quien sube la malla generada en tiempo de ejecución al componente correspondiente.

2. Bibliotecas de terceros

2.1. Resumen

El proyecto **no incorpora ninguna biblioteca de terceros más allá del propio motor Unreal Engine y de un plugin distribuido con él**. No hay código de terceros compilado dentro del repositorio: no existe una carpeta `ThirdParty`, el fichero de reglas de compilación del módulo (`ProceduralEcosystem.Build.cs`) no declara ninguna dependencia externa al motor y no se enlaza ninguna biblioteca estática o dinámica ajena.

Dicho esto, apoyarse en el motor sí es una decisión de dependencia con consecuencias, y por tanto se documenta con el mismo detalle: las tablas siguientes enumeran los módulos del motor declarados explícitamente como dependencias del módulo propio, la función concreta que cumple cada uno y el motivo por el que se usa en lugar de escribir código propio.

2.2. Módulos del motor utilizados

Declarados en `Source/ProceduralEcosystem/ProceduralEcosystem.Build.cs`. Son once módulos públicos —visibles desde las cabeceras del módulo— más uno privado, `ImageWrapper`, que solo se usa en la implementación.

Módulo	Función que cumple en el proyecto	Motivo de la elección
Core	Tipos y contenedores base: <code>TArray</code> , <code>TMap</code> , <code>FVector</code> , <code>FMath</code> , ruido de Perlin, temporización de alta resolución y <code>ParallelFor</code> para el paso paralelo del <i>tick</i> .	Es la base obligatoria de cualquier módulo. Sus contenedores permiten reservar capacidad de forma explícita, que es lo que hace posible reutilizar los búferes entre <i>ticks</i> sin pedir memoria al sistema.
CoreUObject	Sistema de objetos y reflexión: <code>UObject</code> , <code>UPROPERTY</code> , <code>USTRUCT</code> , serialización y punteros débiles/blandos.	Necesario para que los recursos de especie sean editables en el editor y para que el recolector de basura no libere objetos aún en uso.
Engine	Mundo, actores, subsistemas (<code>UTickableWorldSubsystem</code>), componentes de malla instanciada jerárquica, <i>decals</i> , materiales dinámicos y dibujo de depuración.	Los subsistemas de mundo dan una instancia por mundo con ciclo de vida gestionado; sustituirlos por <i>singletons</i> propios obligaría a reproducir a mano ese ciclo de vida.
InputCore	Tipos de entrada; se mantiene por ser dependencia estándar de la plantilla del módulo.	No se hace uso directo. Se conserva porque su coste es nulo y evita fallos de enlace si se añade interacción.
DeveloperSettings	Clase base <code>UDeveloperSettings</code> de la que hereda <code>UEcosystemSettings</code> : publica los parámetros en <i>Project Settings</i> y los persiste automáticamente en <code>DefaultGame.ini</code> .	Da edición en vivo, persistencia y agrupación por categorías sin escribir ni interfaz ni lector de fichero de configuración. Es la razón de que la calibración completa quepa en un fichero de texto versionable.
RenderCore	Tipos de recursos de render y actualización de texturas dinámicas para los mapas de calor.	Requisito para subir la textura del visualizador de campos desde el hilo de juego de forma segura.
RHI	Interfaz de hardware de render: formatos de píxel y consulta de capacidades.	Dependencia obligada de <code>RenderCore</code> para el mismo uso.

Módulo	Función que cumple en el proyecto	Motivo de la elección
Landscape	Declarado como dependencia por la interoperación con el terreno del motor. El código propio no invoca su API : el relieve lo genera y lo muestrea el propio proyecto, y se traslada al ALandscape del nivel importando el PNG de 16 bits que produce <code>Eco.ExportHeightmap</code> .	Se optó por el intercambio mediante fichero porque la creación de un <i>landscape</i> desde código vive en módulos exclusivos del editor cuya API cambia entre versiones del motor. La declaración del módulo es, por tanto, revisable: hoy no aporta símbolos al binario.
ProceduralMeshComponent	Componente que acepta búferes de vértices, triángulos, normales, tangentes, cuatro juegos de coordenadas de textura y color por vértice generados en tiempo de ejecución. Es el que muestra los árboles <i>hero</i> .	Alternativa propia inviable: implicaría escribir un <i>scene proxy</i> , gestionar los búferes en la GPU y su sincronización entre hilos. El componente del motor lo resuelve y admite los cuatro juegos de coordenadas de textura y el color por vértice en los que se empaquetan los datos de viento.
MeshDescription StaticMeshDescription	Descripción de malla independiente del hardware, usada para convertir la geometría procedural en UStaticMesh horneados por arquetipo.	Es la vía admitida para construir mallas estáticas en tiempo de ejecución con generación de tangentes, normales y particiones por material; hacerlo a mano exigiría replicar el formato interno del motor.
ImageWrapper	Codificación a PNG de 16 bits en la exportación del mapa de alturas (<code>Eco.ExportHeightmap</code>).	Escribir un codificador PNG propio —con su compresión <i>deflate</i> y sus sumas de comprobación— sería trabajo ajeno al objetivo del proyecto y con más superficie de error que valor. Es dependencia privada porque no aparece en ninguna cabecera pública.

FUENTES Y DOCUMENTACIÓN

Unreal Engine se distribuye bajo la *Unreal Engine End User License Agreement* y su código fuente está disponible en github.com/EpicGames/UnrealEngine (requiere vincular la cuenta de Epic Games con GitHub). La documentación de referencia de los módulos citados se encuentra en dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-api-reference, y la guía de programación general en dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine.

2.3. Plugins del motor

Habilitados en `ProceduralEcosystem.uproject`. Ambos se distribuyen con Unreal Engine, por lo que no hay nada que descargar ni instalar aparte:

Plugin	Ámbito	Función y motivo
ProceduralMeshComponent	Ejecución y editor	Aporta el componente de malla procedural descrito en la tabla anterior. Sin él, los árboles <i>hero</i> no se pueden dibujar.
ModelingToolsEditorMode	Solo editor (<code>TargetAllowList: Editor</code>)	Conjunto de herramientas de edición de geometría del editor. No interviene en la ejecución: está restringido al editor mediante <code>TargetAllowList</code> , de modo que no forma parte de una compilación empaquetada del juego.

2.4. Herramientas externas de apoyo

No son bibliotecas —no se enlazan ni forman parte del binario—, pero sí son dependencias del flujo de trabajo y se documentan aquí para que el listado sea completo:

Herramienta	Función	Fuente
Git LFS	Almacenamiento de los recursos binarios del proyecto (.uasset, .umap, texturas). Es obligatorio para obtener una copia funcional del repositorio.	git-lfs.com
PlantUML	Generación de los diagramas UML de este anexo a partir de los ficheros de texto incluidos en el repositorio. No interviene en la compilación ni en la ejecución del proyecto.	plantuml.com

2.5. Recursos artísticos

El proyecto genera por procedimiento toda su geometría y sus campos, pero sí utiliza texturas de mapa de bits para la corteza. En Content/Materials_TreeBark/Textures/ hay nueve juegos de texturas (basecolor, normal y un mapa empaquetado ORMH) que alimentan las instancias del material maestro.

PENDIENTE DE COMPLETAR

Estas texturas son recursos importados, no código. Si proceden de una biblioteca de recursos externa, debe indicarse aquí su procedencia y su licencia; si se han creado para el proyecto, basta con hacerlo constar. Este apartado se deja señalado expresamente porque solo el autor puede aportar ese dato.

2.6. Por qué no se han usado bibliotecas externas

La ausencia de dependencias externas no es casual, sino consecuencia de tres exigencias del proyecto:

- **Reproducibilidad exacta.** El sistema garantiza que una misma semilla produce siempre el mismo bosque, incluso ejecutándose en paralelo. Eso obliga a controlar hasta el último consumo de aleatoriedad, y por eso el generador es propio (EcoRand, *xorshift32* de 32 bits de estado, con un flujo independiente por subsistema) en lugar de una biblioteca de terceros cuyo orden de consumo no se controla. Por el mismo motivo está prohibido en el código de simulación llamar a `FMath::Rand()` o a `rand()`.
- **Lo que hacía falta ya estaba en el motor.** Ruido de Perlin, paralelismo por tareas, contenedores con reserva explícita, reflexión, serialización y codificación de imágenes vienen resueltos y probados. Añadir una biblioteca externa para cualquiera de ellos habría sumado una superficie de integración y de licencias sin ganancia funcional.
- **Los algoritmos del núcleo son el objeto del trabajo.** La colonización del espacio, el modelo de tubería, la extinción de Beer-Lambert, las curvas de nicho y el reparto de recursos son precisamente lo que el proyecto desarrolla y justifica; delegarlos en una biblioteca vaciaría de contenido la aportación propia.

3. Clases

3.1. Alcance, notación y convenios de representación

El proyecto está escrito íntegramente en C++ siguiendo una metodología orientada a objetos, por lo que este apartado se puede abordar de forma directa: no hace falta justificar ninguna omisión. Los diagramas que siguen recogen los aproximadamente sesenta tipos que componen el módulo, agrupados por componente.

Por qué varios diagramas y no uno solo. Un único diagrama con todos los tipos sería ilegible a tamaño de página y, sobre todo, no diría nada: la información útil está en las relaciones dentro de cada componente y en los pocos puntos por los que un componente se comunica con otro. Se presenta por tanto un diagrama por componente, con el mismo nombre y el mismo reparto que en el apartado 1, de modo que ambas vistas se pueden contrastar la una con la otra. Cuando un tipo de otro componente es necesario para entender una relación, aparece como caja sin miembros, a modo de referencia.

Convenios de nomenclatura de Unreal Engine. El prefijo de cada tipo indica su naturaleza y por eso se conserva en los diagramas: U para clases que heredan de UObject (gestionadas por el recolector de basura y visibles para la reflexión), A para actores colocables en el mundo, F para clases y estructuras C++ planas sin reflexión, y E para enumerados.

Estereotipos empleados. Se indica entre comillas angulares la clase base o la naturaleza del tipo: «UTickableWorldSubsystem», «UDeveloperSettings», «UPrimaryDataAsset», «UObject», «AActor», «USTRUCT» (estructura reflejada) y «struct» (estructura plana). El estereotipo «utilidad» corresponde al concepto UML de *utility*: un espacio de nombres de C++ que agrupa funciones libres sin estado y que en UML se representa como una clase cuyas operaciones son todas estáticas. Es el caso de EcoRand, EcoGrid, EcologyRules, EcoVigor, SpaceColonization, TreeMeshBuilder y varios más: son deliberadamente funciones puras, porque así se pueden probar de forma aislada y llamar desde un bucle paralelo sin sincronización.

Relaciones. El rombo relleno indica composición: el todo contiene la parte por valor y su vida es la misma (por ejemplo, UEcosystemSubsystem contiene sus dos poblaciones). El rombo hueco indica agregación: la parte se referencia pero tiene vida propia (por ejemplo, la lista de especies, que son recursos del proyecto). La flecha continua es una asociación navegable mediante puntero, y la flecha discontinua con el estereotipo «use» es una dependencia de uso sin almacenamiento. Las multiplicidades se anotan cuando aportan información.

Selección de miembros. Los atributos y operaciones mostrados son representativos, no exhaustivos: se han elegido los que definen la responsabilidad del tipo o intervienen en alguna relación del diagrama. La lista completa está en las cabeceras, que están documentadas.

3.2. Núcleo

Núcleo (Core): utilidades transversales

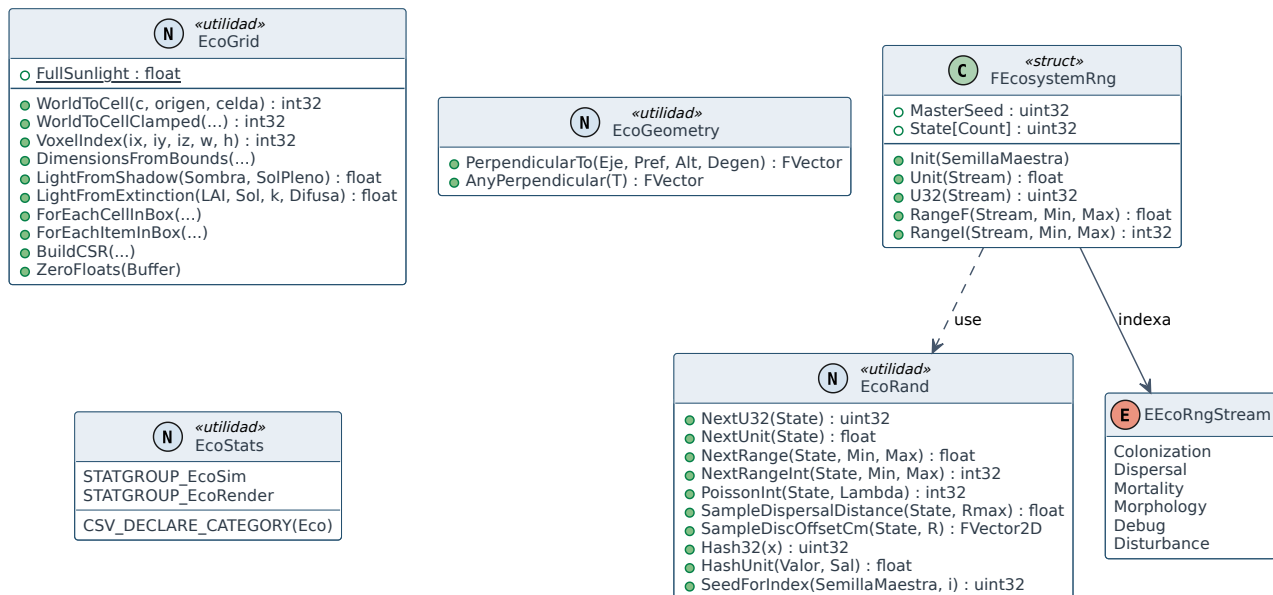


Figura 2. Núcleo: utilidades transversales sin estado.

El componente no contiene ninguna clase con estado propio salvo `FEcosystemRng`, y esa ausencia es intencionada: todo lo que vive aquí lo usan a la vez el hilo de juego, el paso paralelo del *tick* y el generador de geometría, de modo que cualquier estado compartido sería una fuente de indeterminismo.

`EcoRand` implementa un generador *xorshift32* cuyo estado cabe en un `uint32` que se pasa **por referencia explícita** en cada llamada. Ese detalle de firma es lo que permite que cada árbol lleve su propio flujo y que, en consecuencia, el resultado de un *tick* no dependa de qué hilo procesa cada individuo. Sobre esa base se construyen los muestreos de dominio: número de semillas según una distribución de Poisson, desplazamiento uniforme por área dentro de un disco de dispersión y funciones de mezcla de bits (`Hash32`, `HashUnit`) que producen valores estables **sin consumir ningún flujo**, lo que permite al mallador y al render derivar variaciones por individuo sin desplazar la secuencia de la simulación.

`FEcosystemRng` deriva de una única semilla maestra un flujo independiente por subsistema, enumerados en `EEcoRngStream`. Que los flujos estén separados tiene una consecuencia experimental concreta: activar o desactivar el régimen de perturbaciones no desplaza la secuencia de los demás, así que una corrida con claros y otra sin ellos parten del mismo bosque y la comparación mide únicamente el efecto buscado. Por ese motivo el flujo `Disturbance` se añadió al final del enumerado, después de `Debug`.

`EcoGrid` concentra la aritmética de rejilla que comparten las cinco estructuras espaciales del proyecto: conversión de mundo a celda con recorte, indexado lineal de vóxeles, dimensionado a partir de una caja envolvente, recorrido de un bloque de celdas vecinas y construcción del índice espacial CSR mediante *counting sort*. El recorrido tiene un orden de visita fijo (z, después y, después x, todos ascendentes) porque las consultas de vecindad alimentan decisiones de la simulación: el orden forma parte del contrato de determinismo. Aquí vive también la ley de Beer-Lambert que convierte área foliar acumulada en luz disponible.

3.3. Configuración y datos de especie

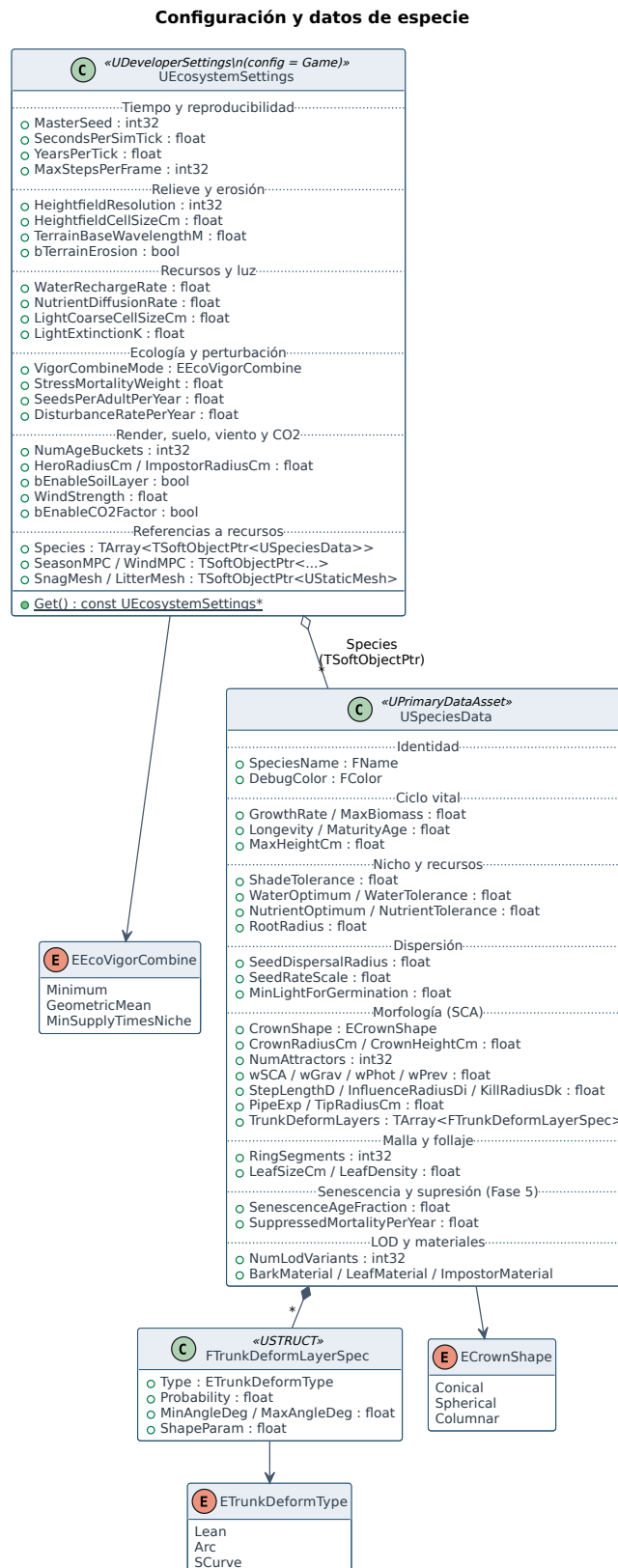


Figura 3. Configuración global y descripción de especies.

La configuración está partida en dos por una razón de diseño: UEcosystemSettings describe **el mundo** (resolución del relieve, tasas de recarga de los campos, radios de nivel de detalle, viento, estaciones, presupuesto de tiempo por

fotograma) y `USpeciesData` describe **un organismo** (longevidad, tolerancia a la sombra, óptimos de nicho, morfología de copa y de tronco). Añadir una especie es, por tanto, crear un recurso y añadirlo a una lista, sin recompilar ni tocar código.

La relación entre ambos es una agregación mediante `TSoftObjectPtr`: la configuración no obliga a cargar los recursos de especie al arrancar el motor. Quien los resuelve es el subsistema de simulación, una sola vez al comenzar la partida, y guarda el resultado en una caché; sin ella habría que resolver la referencia por árbol y por *tick*, lo que a escala de miles de individuos sería el principal coste del bucle.

La composición de `FTrunkDeformLayerSpec` merece un comentario porque es lo que distingue este modelo de una simple tabla de parámetros: una especie no declara «cómo es su tronco», sino la **distribución** de troncos que produce. Cada capa lleva su propia probabilidad y su rango de ángulos, y cada árbol tira los dados de forma independiente por cada una, de modo que una capa `Arc` con probabilidad 0,4 da cuatro individuos arqueados de cada diez —con ángulos y azimuts distintos— y seis rectos.

3.4. Terreno, campos de recurso y luz

(Figura 4.) `FField2D` es la única abstracción de campo escalar bidimensional del proyecto: guarda dimensiones, tamaño de celda, origen en el mundo y los datos en un vector plano, y ofrece muestreo bilineal, normalización y generación en paralelo. Los tres campos del dominio —relieve, agua y nutrientes— **lo componen en lugar de heredar de él**. Es una decisión deliberada: comparten representación pero no semántica, y la composición mantiene una interfaz de muestreo con nombre propio en cada dominio (`SampleHeight`, `SampleWater`, `SampleNutrient`), que se lee mejor en el punto de uso y evita que un campo se pase por otro.

El relieve se genera en dos fases claramente separadas. Primero, ruido fractal —`EcoNoise` implementa *fBm*, multifractal híbrido y multifractal de crestas, con desplazamiento del dominio y recorte de octavas por el criterio de Nyquist para que no se pida al ruido más detalle del que la rejilla puede representar—. Después, erosión: `TerrainErosion` aplica erosión térmica por ángulo de talud y erosión hidráulica por gotas con inercia, capacidad de sedimento y evaporación. Ambas etapas se parametrizan con estructuras propias (`FThermalParams`, `FHydraulicParams`) agregadas en `FTerrainGenParams`, lo que permite reproducir un relieve idéntico a partir de la semilla y estos parámetros.

`FWaterField` no inventa la humedad: la deriva del relieve mediante el índice topográfico de humedad, de manera que los fondos de vaguada acumulan agua y las crestas quedan secas. Esa dependencia —`BakeFromHeightField`— es la que hace que el paisaje ecológico y el paisaje geométrico sean coherentes entre sí.

`FResourcePool` añade a los campos base la dimensión temporal: mantiene **dos** campos, el actual y el siguiente. El paso paralelo del *tick* lee siempre del actual y escribe en el siguiente, y al final se intercambian. Sin ese doble búfer, el resultado de un árbol dependería de cuántos vecinos hubieran consumido ya su recurso, es decir, del orden de proceso.

`FLightFieldCoarse` es una rejilla de vóxeles de varios metros que almacena **área foliar**, no luz. La luz se obtiene al muestrear, aplicando la ley de Beer-Lambert sobre el área foliar acumulada por encima del punto, más un suelo de luz difusa. La diferencia importa: con una resta lineal, en cuanto la sombra supera la luz plena el resultado se clava en cero y todas las especies quedan igualadas justo en la sombra profunda, que es el único sitio donde una especie tolerante debería ganar. La exponencial es asintótica y conserva el orden entre especies a cualquier densidad de dosel. La rejilla puede además referirse a la altura sobre el terreno en lugar de a la *Z* absoluta, lo que evita que en laderas pronunciadas las copas caigan en capas equivocadas.

Terreno, campos de recurso y luz

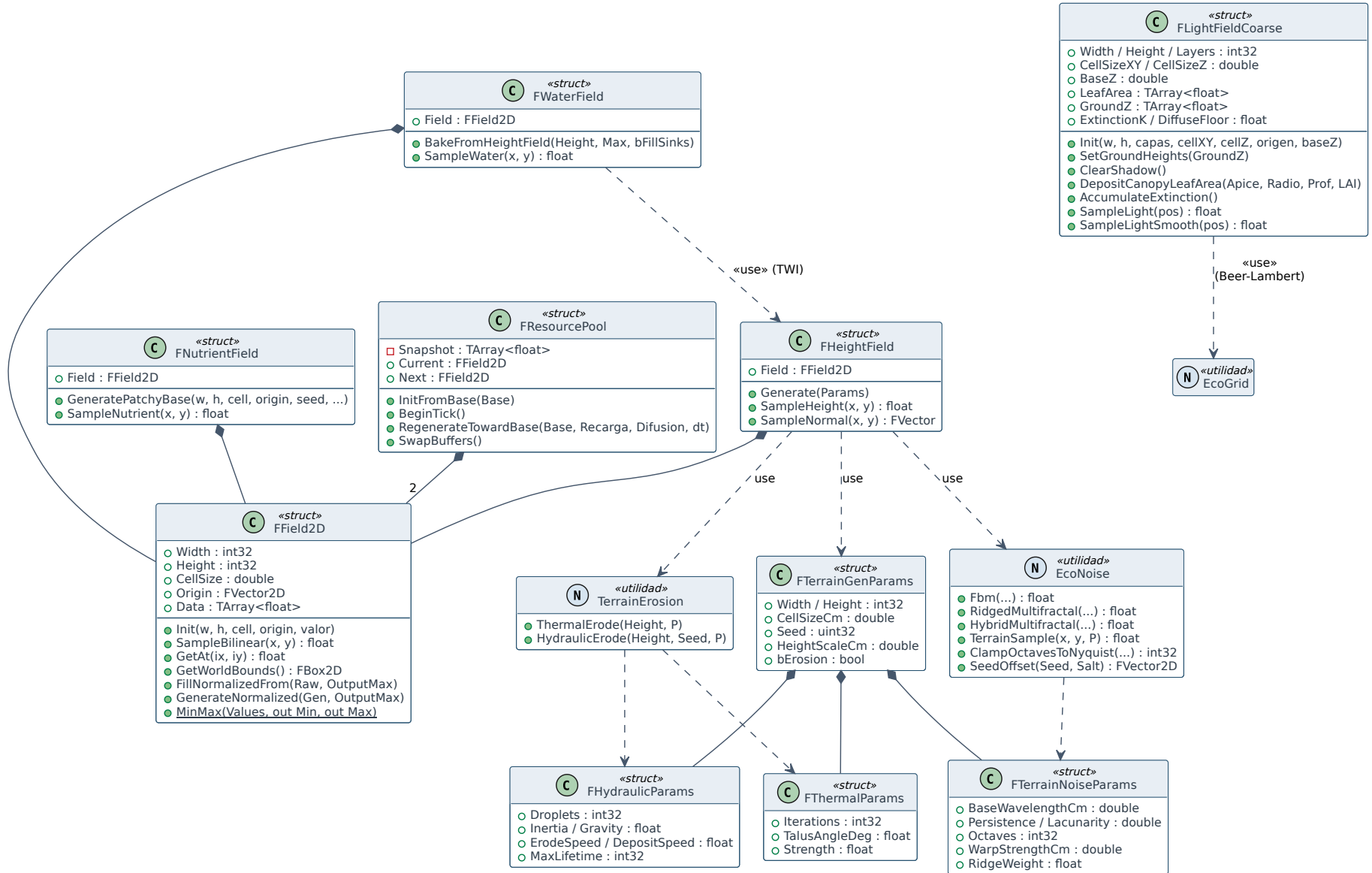


Figura 4. Terreno, campos de recurso y luz.

3.5. Simulación

(Figura 5.) `UEcosystemSubsystem` hereda de `UTickableWorldSubsystem`, lo que le da una instancia por mundo con ciclo de vida gestionado por el motor y un *tick* propio. Concentra todo el estado del ecosistema por composición y expone hacia fuera únicamente consultas de solo lectura, además de las órdenes de control que se manejan desde la consola.

Población en formato SoA. `FTreePopulation` no es un vector de objetos «árbol»: es un conjunto de vectores paralelos, uno por atributo. Cada etapa del *tick* recorre solo los vectores que necesita, con lo que se aprovecha la línea de caché en lugar de arrastrar campos que no se van a leer. El precio es que hay que mantener todos los vectores alineados, y por eso la estructura ofrece un recorrido genérico (`ForEachArray`) que aplica la misma operación a todos: añadir un atributo nuevo es tocar un solo sitio, y la comprobación `AllArraysHaveNum` detecta cualquier desalineación.

Doble búfer de población. El subsistema mantiene `Agents_Read` y `Agents_Write`. El paso paralelo lee del primero —una instantánea inmutable durante todo el paso— y cada tarea escribe únicamente en su porción del segundo. Al terminar se compactan los muertos y se intercambian. Es el mismo principio que en los depósitos de recurso y por el mismo motivo: eliminar toda dependencia del orden de proceso.

Espacio de trabajo por tarea. `FTickScratch` es la pieza que permite paralelizar sin sincronización. Cada tarea acumula en su propio ejemplar las modificaciones que querría hacer sobre estructuras compartidas: consumos de agua y nutrientes como listas **dispersas** de pares (celda, cantidad), semillas producidas, pulsos de muerte y contadores demográficos. Terminado el paso paralelo, una reducción **serial** vuelca todos los ejemplares en un orden fijo. La representación dispersa sustituyó a un campo denso por tarea, que a resoluciones altas reservaba memoria proporcional al número de tareas por el número de celdas del mapa. Los ejemplares son persistentes entre *ticks*: se reinician conservando la capacidad, de modo que una oleada de germinación no vuelve a pedir memoria.

Índice espacial. `FSpatialHash` se reconstruye al principio de cada *tick* con las posiciones vigentes mediante *counting sort*, en tiempo lineal y con orden de salida determinista. Lo consultan el espaciado mínimo de germinación y las inhibiciones entre coespecíficos.

Eventos de muerte. `FTreeDeathEvent` se acumula en un anillo circular de tamaño fijo con un contador monótono. Los consumidores —la capa de suelo— guardan su propio cursor y piden «lo ocurrido desde X». El anillo se dimensiona una sola vez al arrancar, y no en cada muerte, porque los ajustes son editables en vivo y un cambio de tamaño a mitad de partida haría que escritor y lector indexaran de forma distinta.

Instrumentación. `FEcoTickProfile` mantiene una media exponencial del coste de cada etapa del *tick*, y `FEcoDemoSample` registra una fila por especie cada cierto número de *ticks* con recuentos, vigor medio, reparto del recurso limitante y flujos de nacimientos y muertes. Ninguna de las dos consume aleatoriedad ni modifica estado, de modo que tenerlas activadas no altera la evolución de la partida.

Simulación: el subsistema y sus estructuras de estado

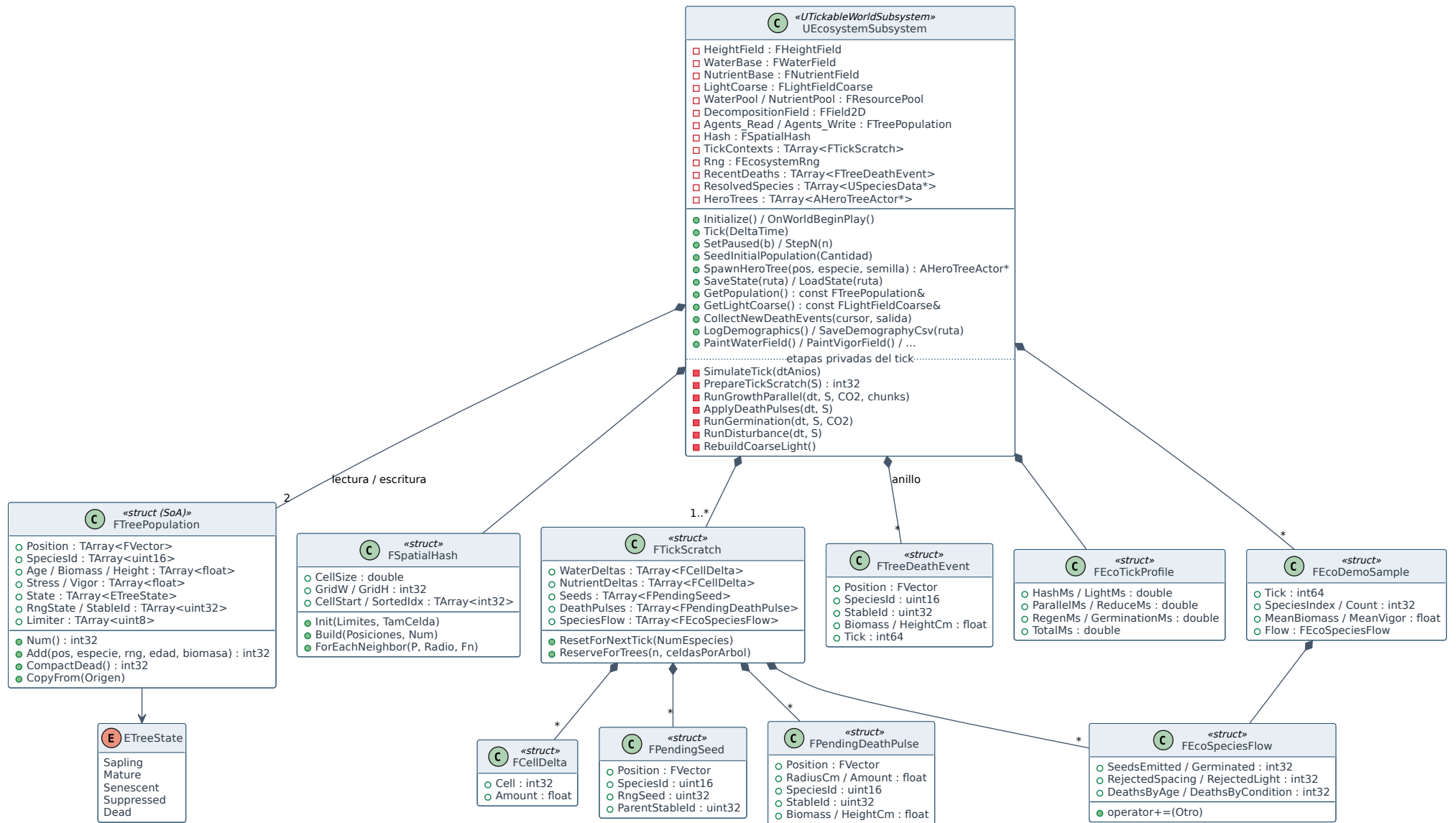


Figura 5. Simulación: el subsistema y sus estructuras de estado.

3.6. Ecología

Ecología: reglas del modelo biológico

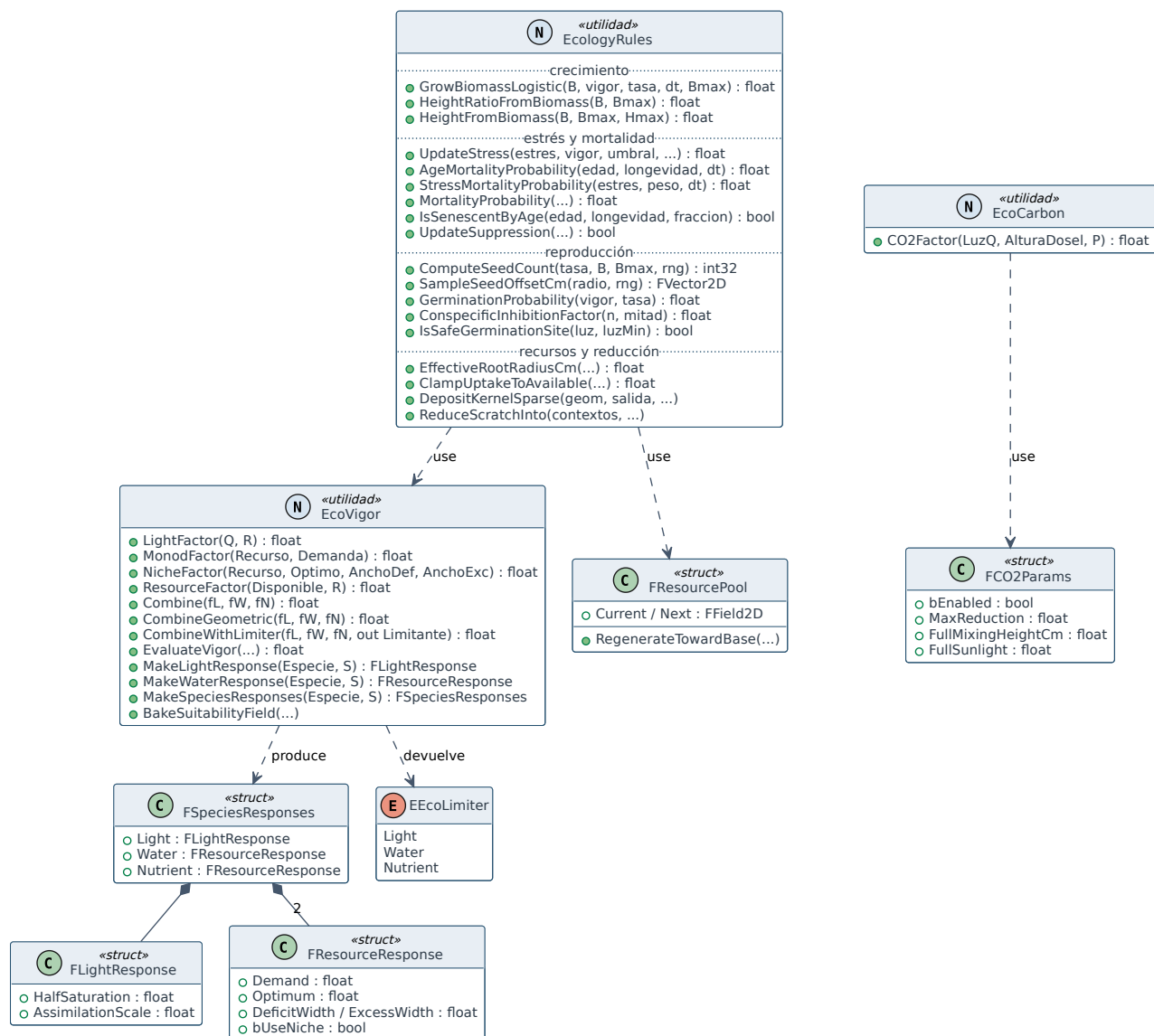


Figura 6. Ecología: reglas del modelo biológico.

(Figura 6.) Este componente es, casi en su totalidad, funciones puras agrupadas en dos espacios de nombres. No es una elección estilística: al no tener estado, se pueden llamar desde el interior del bucle paralelo sin ninguna sincronización y se pueden probar una a una con valores concretos, que es exactamente lo que hacen las pruebas automáticas del proyecto.

EcologyRules contiene el ciclo vital: crecimiento logístico modulado por el vigor, alometría que deriva la altura de la biomasa, acumulación y recuperación de estrés con histéresis, probabilidad de muerte compuesta a partir de riesgos independientes por edad y por condición, transición a senescencia y a supresión, número de semillas por adulto y año, dispersión, probabilidad de germinación e inhibición por densidad de coespecíficos. Incluye también el depósito disperso del núcleo de consumo de recursos y la reducción de los espacios de trabajo.

EcoVigor resuelve la pregunta central del modelo: cuánto vale este sitio para esta especie. Lo hace en dos niveles. Los factores individuales convierten la disponibilidad de cada recurso en un valor normalizado —saturación para la

luz, cinética de Monod para la demanda, y campanas asimétricas de nicho con anchuras de déficit y de exceso distintas— y la combinación los reduce a un vigor único. Se ofrecen tres modos de combinación seleccionables desde la configuración, y la variante `CombineWithLimiter` devuelve además **qué** recurso está limitando. Ese dato sale gratis, porque el mínimo ya se calculaba, y es lo que permite diagnosticar por qué una especie se está extinguiendo en lugar de limitarse a constatarlo.

Las estructuras `FLightResponse` y `FResourceResponse`, agrupadas en `FSpeciesResponses`, son curvas ya resueltas: se construyen **una vez por especie y por tick**, fuera del bucle paralelo. Antes se rehacían por individuo, lo que además obligaba a leer el objeto de especie desde dentro del bucle. `EcoCarbon` añade un multiplicador que reduce la asimilación en el interior del dosel, donde la mezcla de aire es menor.

3.7. Geometría procedural (I): esqueleto

(Figura 7.) `AHeroTreeActor` es el único actor del componente y actúa como fachada: reúne por composición las cuatro estructuras que intervienen en la generación —esqueleto, rejilla fina de luz, nube de atractores y malla— y ofrece una operación `Generate` que recibe la especie, la semilla, el campo de luz del bosque y la posición de la base del tronco.

El algoritmo vive en `SpaceColonization`, no en el actor. La separación permite generar árboles sin instanciar actores, que es justo lo que necesita la biblioteca de arquetipos del componente de render al hornear mallas, y permite además probar el algoritmo en el marco de automatización.

El proceso encadena cuatro estructuras. `FAttractorCloud` siembra puntos dentro de la envolvente de copa que declara la especie —cónica, esférica o columnar, con ruido en el radio y sesgo vertical— y los indexa en una rejilla CSR para poder consultar los que están al alcance de un nodo sin recorrerlos todos. `FTreeSkeleton` es un árbol de nodos `FBranchNode` con padre, dirección, profundidad, radio y unas banderas que marcan si el nodo pertenece al eje principal. `FTreeLightGridFine` es una rejilla de vóxeles de unos treinta centímetros, local al árbol, que se siembra con la sombra de los vecinos tomada del campo de luz gruesa: esa es la conexión micro-macro que hace que un árbol crecido a la sombra de otro se desarrolle de forma asimétrica. Su gradiente alimenta el fototropismo y su umbral de sombra, la autopoda.

La dirección de crecimiento de cada nodo se obtiene mezclando cuatro términos con los pesos que declara la especie: la atracción de los puntos no colonizados, la gravedad, el gradiente de luz y la inercia de la dirección anterior. Sobre el esqueleto ya crecido se aplican después dos pasadas independientes: `ComputeRadii`, que reparte grosor según el modelo de tubería, y `TrunkDeformer`, que superpone las capas de deformación —inclinación rígida, arqueado progresivo o serpenteo— que le hayan tocado a ese individuo.

Geometría procedural (I): esqueleto por colonización del espacio

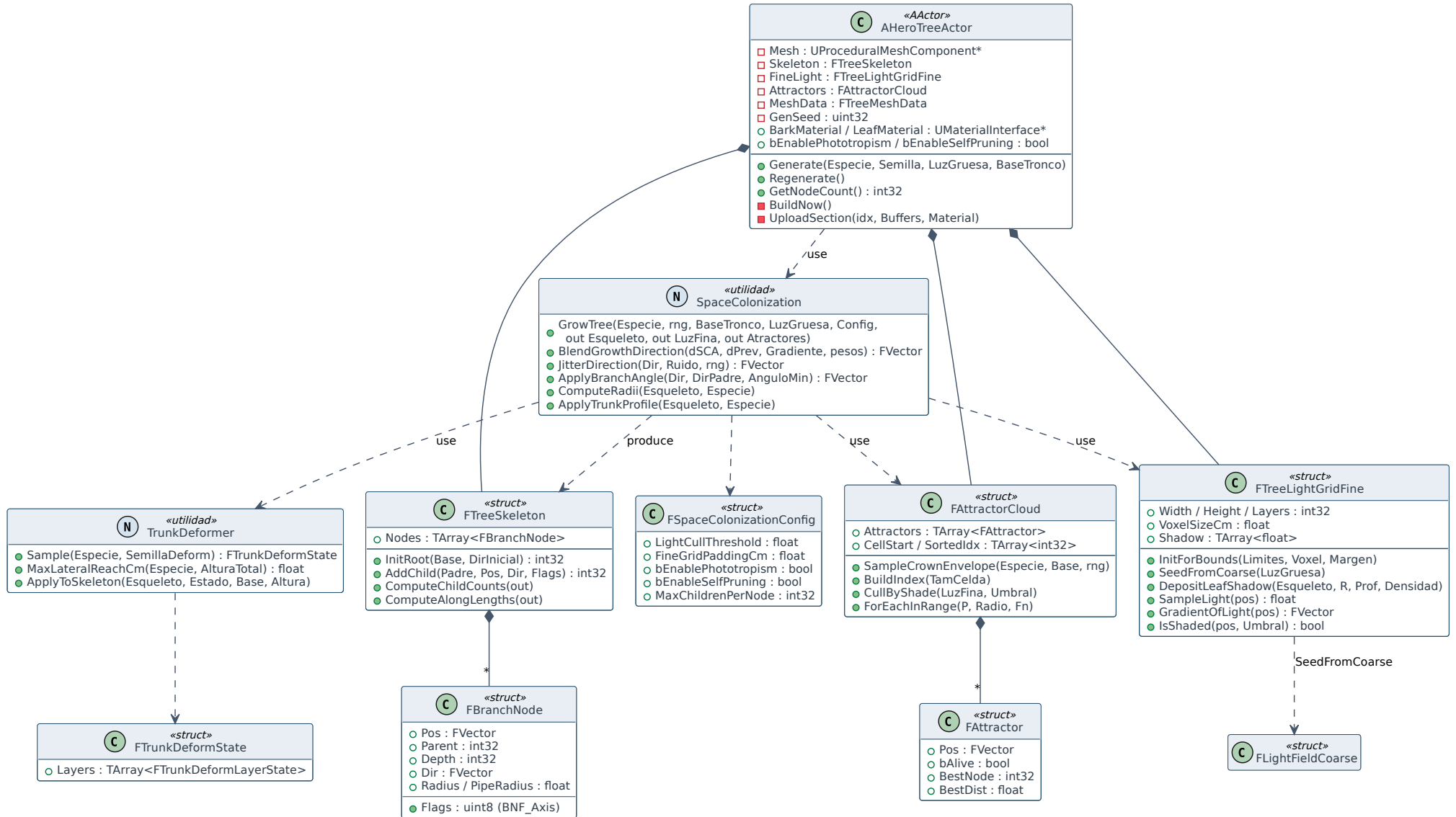


Figura 7. Geometría procedural (I): esqueleto por colonización del espacio.

3.8. Geometría procedural (II): mallado, follaje y viento

La malla se construye en dos secciones separadas —leño y hojas—, porque cada una lleva su propio material. Eso es lo que representa `FTreeMeshData`, que compone dos `FTreeMeshBuffers`.

`FTreeMeshBuffers` es un contenedor de vectores paralelos con la misma filosofía SoA de la población, pero aquí el detalle relevante son los canales adicionales. Además de posición, normal, tangente y coordenadas de textura, cada vértice lleva tres juegos más de coordenadas y un color. Ahí se empaquetan el pivote local de la rama, el nivel de ramificación, el peso de balanceo, un desfase por rama, la oclusión ambiental de copa y una variación de tinte. Con esos datos, el material anima el viento en la GPU mediante desplazamiento de posición del mundo, sin que la CPU intervenga por árbol. El empaquetado está encapsulado en `SetWindVertex` por un motivo concreto: es el contrato con el material, y si el árbol *hero* y la instancia horneada lo escribieran cada uno por su lado y divergieran, dejarían de moverse igual.

`FTreeWindData` precalcula esos canales una sola vez por árbol a partir del esqueleto, junto con información derivada que necesitan tanto el mallador como el follaje —raíz de rama, número de hijos por nodo y longitud acumulada a lo largo de la rama—, evitando que cada uno la recalculen. `TreeFoliage` coloca las hojas sobre las ramas terminales siguiendo la filotaxis de la especie y orientándolas hacia la luz, y escribe directamente en los búferes de hoja.

Geometría procedural (II): mallado, follaje y viento

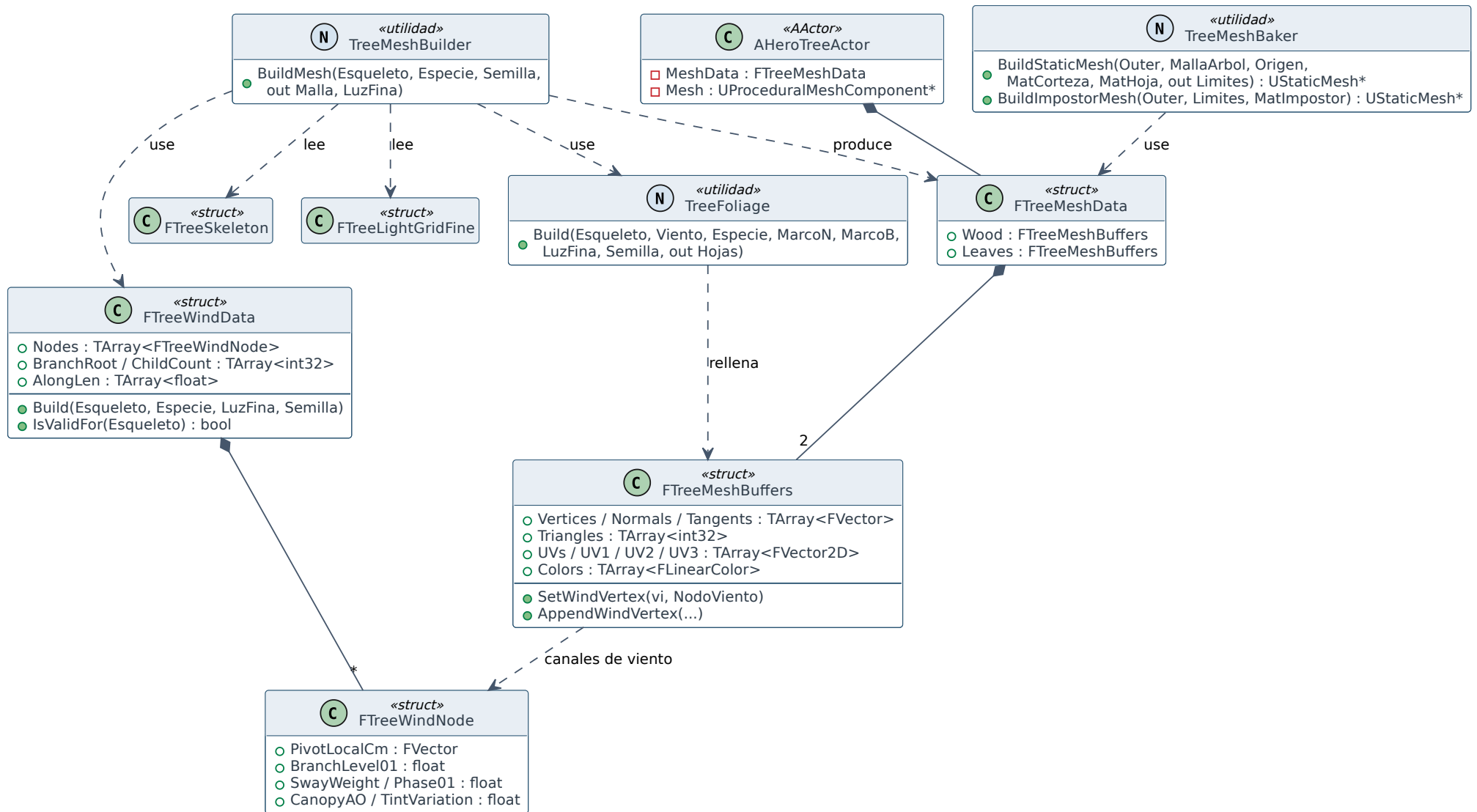


Figura 8. Geometría procedural (II): mallado, follaje y canales de viento.

3.9. Render (I): gestor de nivel de detalle

(Figura 9.) `UTreeRenderSubsystem` resuelve el problema de dibujar miles de árboles cuyo número y tamaño cambian continuamente. Cada árbol vivo recibe un nivel de representación (`ETreeRenderTier`): geometría única generada en vivo para las decenas más cercanas, malla instanciada para los miles intermedios, tarjetas cruzadas para el campo lejano y ninguna representación fuera del radio de corte. Conviene subrayar que un árbol sin representación **se sigue simulando**: el nivel de detalle es una decisión de vista y no afecta al modelo.

El estado por árbol se guarda en un mapa indexado por `StableId` y no por índice de población, porque los índices se reordenan en cada *tick* al compactar los muertos. Ese detalle es el que evita que la representación de un árbol acabe colocada sobre otro.

Las transiciones entre niveles se estabilizan con histéresis, implementada en el espacio de nombres `TreeArchetype`: un árbol solo cambia de tramo de tamaño cuando supera el umbral más un margen, de modo que un individuo situado justo en la frontera no oscila entre dos mallas fotograma tras fotograma. El mismo espacio de nombres deriva de forma estable —a partir del identificador del árbol y sin consumir aleatoriedad— la variante morfológica, el giro sobre su eje y el *jitter* de escala, de manera que dos árboles del mismo arquetipo no se vean como copias exactas.

Las operaciones sobre los componentes instanciados no se aplican de una en una: se acumulan en `FPendingComponentOps` —altas, bajas, actualizaciones de transformación y de datos personalizados— y se vuelcan por lotes al final del fotograma. Los valores continuos que van a los datos personalizados (sequedad y apertura de copa) se cuantizan en dieciséis bandas para no reescribir todas las instancias cuando cambian unas milésimas. La generación de árboles *hero* también está limitada por fotograma mediante una cola.

Render (I): gestor de nivel de detalle y arquetipos

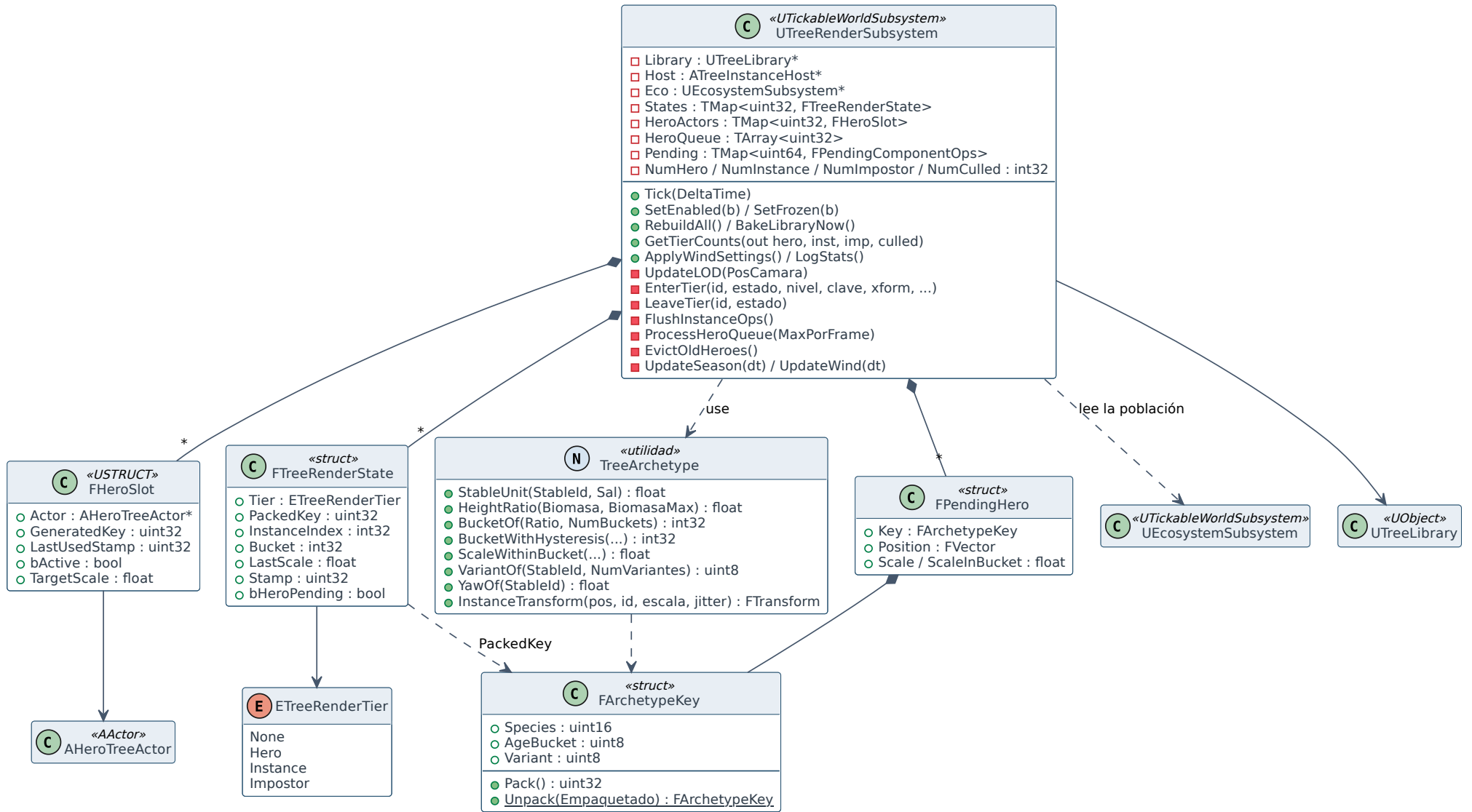


Figura 9. Render (I): gestor de nivel de detalle y arquetipos.

3.10. Render (II): biblioteca de arquetipos

La idea que sostiene todo el nivel de detalle intermedio es el **arquetipo**: en lugar de una malla por árbol, una malla por combinación de especie, tramo de tamaño y variante morfológica. `FArchetypeKey` empaqueta esa tripleta en un `uint32` que sirve directamente de clave de mapa y de identificador comparable.

`UTreeLibrary` mantiene una entrada por arquetipo con su malla de leño, su malla de *impostor*, los componentes de malla instanciada jerárquica correspondientes y la correspondencia entre índices de instancia e identificadores de árbol. El horneado es perezoso y con presupuesto: cuando se pide un arquetipo que aún no existe, se encola y se procesa un número limitado de arquetipos por fotograma, de modo que la aparición de una especie nueva no produce una pausa visible. La cola usa un cursor de cabeza en lugar de extraer del principio del vector, porque drenar n elementos desplazando el resto sería cuadrático.

`TreeMeshBaker` es el puente entre los dos mundos: toma los búferes procedurales que produce el mallador y construye con ellos un `UStaticMesh` con sus dos particiones de material, además de la malla de tarjetas cruzadas del *impostor* a partir de la caja envolvente. `ATreeInstanceHost` es un actor transitorio que existe únicamente para alojar los componentes instanciados; ofrece solo operaciones estáticas de creación.

Render (II): biblioteca de arquetipos y horneado de mallas

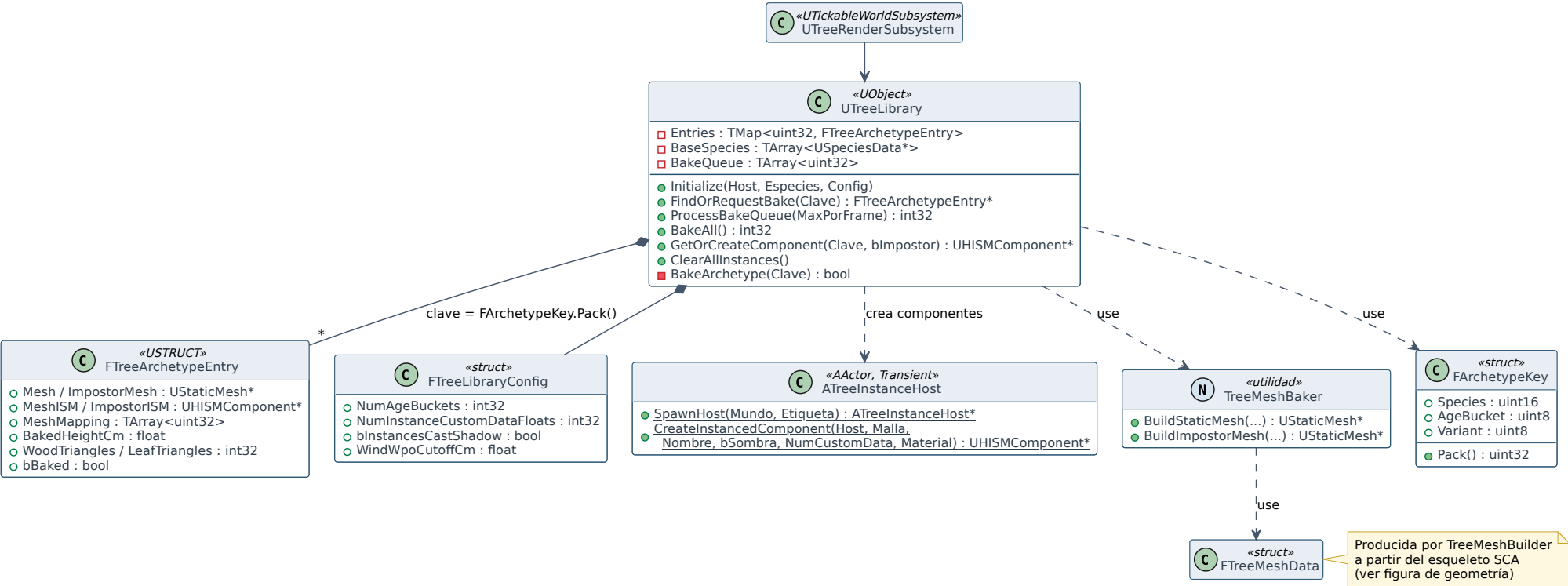


Figura 10. Render (II): biblioteca de arquetipos y horneado de mallas.

3.11. Capa de suelo, instrumentación y depuración

(Figura 11.) `UTreeSoilSubsystem` traduce las muertes en materia visible. Consume el anillo de eventos con su propio cursor y, por cada árbol muerto, encola un tocón y una cantidad de hojarasca proporcional. Cada tocón (`FSoilSnag`) recorre una máquina de estados —en pie, cayendo, tronco tumbado y retirado— con tiempos configurables, y su transformación se recalcula solo mientras está en movimiento. Tocones y hojarasca se gestionan como anillos de tamaño acotado: al llegar al máximo se reutiliza la instancia más antigua, lo que da un coste de memoria constante independientemente de cuántos siglos se simulen.

`UEcoFrameProfiler` mide el fotograma completo y lo relaciona con las dos fuentes de coste propias: el tiempo del *tick* de simulación y el del re-nivelado de detalle. Mantiene una ventana deslizante para calcular percentiles y puede capturar durante un intervalo a un CSV con una fila por fotograma —población, coste, reparto por niveles—, que es directamente el material de las gráficas de resultados.

`UFieldVisualizer` convierte cualquier campo escalar en una textura dinámica con una rampa de color; el subsistema de simulación la proyecta sobre el terreno con un *decal*. Sobre esa base están contruidos los mapas de calor de agua, nutrientes, luz, vigor y descomposición. Completan el componente la auditoría de especies, que compara los rasgos declarados y sus derivados, y la exportación del mapa de alturas a PNG de 16 bits, cuyo formato coincide vértice a vértice con el que espera la importación de *landscape* del editor.

Capa de suelo, instrumentación y depuración

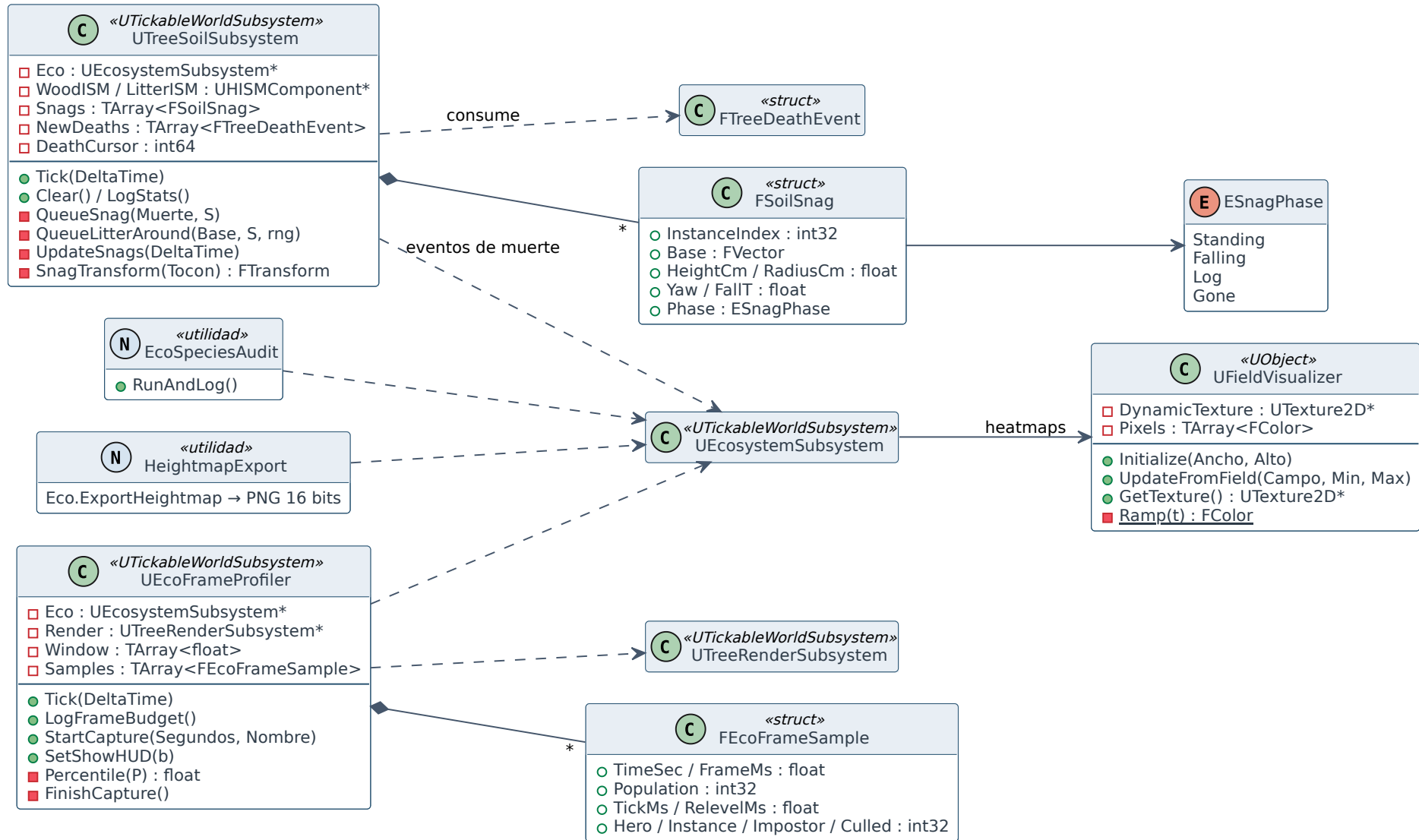


Figura 11. Capa de suelo, instrumentación y depuración.

4. Requisitos para usar el proyecto

El proyecto se ha desarrollado y probado sobre **Windows 10/11 de 64 bits**, que es la plataforma de referencia. La configuración del proyecto incluye también los formatos de sombreado de Linux (Vulkan SM6) y macOS (Metal SM6), pero ninguna de esas dos plataformas se ha verificado; las instrucciones de este apartado son las de Windows.

4.1. Requisitos de hardware

Elemento	Mínimo	Recomendado y motivo
Procesador	4 núcleos, 64 bits	8 núcleos o más. El paso de crecimiento del <i>tick</i> se reparte con <code>ParallelFor</code> , de modo que el número de núcleos se traduce directamente en tamaño de población simulable a la misma velocidad. La compilación del módulo también escala con los núcleos.
Memoria	16 GB	32 GB. El editor de Unreal Engine con el proyecto abierto y compilando sombreadores es el consumidor principal, no la simulación.
Tarjeta gráfica	Compatible con DirectX 12 y <i>Shader Model 6</i>	Tarjeta con trazado de rayos por hardware (DXR). El proyecto tiene activados Lumen con trazado por hardware, Nanite y mapas de sombras virtuales. Sin DXR el proyecto funciona igualmente , porque Lumen recurre a su modo por software, pero el aspecto de la iluminación cambia.
Disco	~60 GB libres	El motor ocupa la mayor parte; el proyecto en sí es pequeño, pero la caché de datos derivados y los intermedios de compilación crecen varios gigabytes.

4.2. Software necesario

Software	Versión	Necesario para
Unreal Engine	5.7	Es la versión declarada en <code>ProceduralEcosystem.uproject</code> ("EngineAssociation": "5.7") y la que fijan los <i>targets</i> de compilación (<code>EngineIncludeOrderVersion.Unreal5_7</code> , <code>BuildSettingsVersion.V6</code>). Con una versión anterior el proyecto no compila. Con una posterior, el editor ofrecerá convertirlo, lo que crea una copia y puede requerir ajustes: debe usarse 5.7 .
Visual Studio 2022	17.8 o superior	Compilador de C++ y depurador. El repositorio incluye un fichero <code>.vsconfig</code> con la lista exacta de componentes; Visual Studio la detecta al abrir la solución y ofrece instalar lo que falte.
Git	2.30 o superior	Obtener el repositorio.
Git LFS	3.x	Obligatorio. Todos los recursos binarios (<code>.uasset</code> , <code>.umap</code> y las texturas) se almacenan con <i>Large File Storage</i> . Sin él, la copia de trabajo contendrá ficheros de texto con punteros en lugar de recursos, y el editor no podrá abrir el proyecto.
.NET Framework	4.6.2 (paquete de destino)	Lo requiere <i>UnrealBuildTool</i> . Se instala como componente de Visual Studio.

Componentes de Visual Studio declarados en `.vsconfig`:

- **Cargas de trabajo:** Desarrollo de escritorio con C++ (NativeDesktop), Desarrollo de juegos con C++ (NativeGame), Desarrollo de escritorio con .NET (ManagedDesktop) y el editor base (CoreEditor).
- **Integración con Unreal Engine:** Component.Unreal.Ide y Component.Unreal.Debugger, que aportan la integración del editor y los visualizadores de depuración de los tipos del motor.
- **Compiladores:** conjuntos de herramientas MSVC v14.38 y v14.44 (x86/x64 y ATL) y Clang para Windows.
- **SDK:** Windows 11 SDK 10.0.22621 y el paquete de destino de .NET Framework 4.6.2.

4.3. Instalación paso a paso

```
# 1. Instalar Git LFS una sola vez por máquina (antes del primer clonado)
git lfs install

# 2. Clonar el repositorio; LFS descarga los binarios automáticamente
git clone https://github.com/Izana-L/ProceduralEcosystem.git
cd ProceduralEcosystem

# 3. Comprobar que los recursos son binarios reales y no punteros de LFS
git lfs ls-files | more
```

1. **Instalar Unreal Engine 5.7** desde Epic Games Launcher (pestaña *Unreal Engine* → *Library*). La instalación registra las asociaciones de fichero necesarias para el paso 5.
2. **Instalar Visual Studio 2022** con los componentes indicados. Si ya está instalado, basta con abrir el *Visual Studio Installer*, elegir *Modificar* y usar la opción de importar configuración señalando el fichero `.vsconfig` del repositorio.
3. **Instalar Git y Git LFS** y ejecutar `git lfs install`.
4. **Clonar el repositorio** con las órdenes anteriores. Si el repositorio se hubiera clonado antes de instalar LFS, se corrige con `git lfs install` seguido de `git lfs pull`.
5. **Generar los ficheros de proyecto:** pulsar con el botón derecho sobre `ProceduralEcosystem.uproject` y elegir *Generate Visual Studio project files*. Se creará `ProceduralEcosystem.sln` junto a las carpetas `Intermediate/` y `Binaries/`, ninguna de las cuales se versiona.
6. **Compilar:** abrir la solución, seleccionar la configuración **Development Editor** y la plataforma **Win64**, y compilar. La primera compilación completa tarda varios minutos.
7. **Abrir el proyecto** haciendo doble clic en el `.uproject` o iniciando la depuración desde Visual Studio. El primer arranque compila los sombreadores del motor y puede tardar bastante; es un coste que solo se paga una vez por máquina.

NO BORRAR DIRECTORY.BUILD.TARGETS

El fichero `Directory.Build.targets` de la raíz vacía las propiedades `IncludePath` y `LibraryPath` de MSBuild antes de que se establezcan las variables de entorno de compilación. Es una solución a un fallo de compilación por rutas de inclusión excesivamente largas, que se manifiesta cuando el proyecto o el motor están instalados en rutas profundas. Si se elimina el fichero, la compilación puede fallar con errores del compilador difíciles de relacionar con su causa. Aun así, la recomendación habitual sigue siendo mantener rutas cortas para el motor y para el proyecto.

4.4. Configuración del proyecto

Con la configuración incluida en el repositorio el proyecto arranca y funciona sin ajustes adicionales. Lo que sigue describe dónde se cambia cada cosa y qué queda desactivado por falta de recursos.

Parámetros del ecosistema. Están en *Edit* → *Project Settings* → *Game* → *Procedural Ecosystem*. La clase que los publica es `UEcosystemSettings`, un `UDeveloperSettings` con `config = Game`, de modo que cualquier cambio hecho en el editor se escribe en `Config/DefaultGame.ini` y queda versionado. Ese fichero es la **tabla completa de calibración**: fija todas las claves con efecto ecológico, incluso las que coinciden con el valor por defecto de C++, para que un cambio de valor por defecto en el código no altere la ecología en silencio. La cobertura se comprueba con el comando `Eco.Config.Auditar`, que avisa tanto de propiedades ausentes del `.ini` como de claves huérfanas.

PARÁMETROS QUE NO CONVIENE TOCAR

Los valores de la sección de relieve (`HeightfieldResolution`, `HeightfieldCellSizeCm`, `HeightScaleCm`, octavas, longitud de onda, persistencia, lacunaridad y parámetros de erosión) están fijados a propósito: el campo de agua es el índice topográfico de humedad **de ese relieve concreto** y toda la calibración de óptimos hídricos de las especies se apoya en sus percentiles. Cambiar cualquiera de ellos invalida esa calibración sin que nada avise.

Funcionalidades que dependen de recursos aún no creados. El fichero `DefaultGame.ini` deja comentadas seis referencias a recursos que deben crearse en el editor. Mientras no se asignen, la funcionalidad correspondiente simplemente no se aplica: el proyecto arranca y funciona igual, solo que sin viento, sin ciclo estacional y sin capa de suelo visible.

Clave	Recurso que hay que crear	Efecto mientras no se asigne
<code>SeasonMPC</code>	<i>Material Parameter Collection</i> de estación	El ciclo estacional no llega a los materiales.
<code>WindMPC</code>	<i>Material Parameter Collection</i> de viento	Los árboles no se mueven, aunque los canales de viento sí estén en la malla.
<code>SnagMesh</code>	Malla estática de tocón (sirve <code>/Engine/BasicShapes/Cylinder</code>)	La capa de suelo no coloca ni tocones ni hojarasca.
<code>LitterMesh</code>	Malla estática plana (sirve <code>/Engine/BasicShapes/Plane</code>)	
<code>SnagMaterial</code>	Material de madera muerta	
<code>LitterMaterial</code>	Material de hojarasca	

Especies activas. La lista `Species` del `.ini` contiene las tres especies de laurisilva de `Content/Species/Garajonay/`. El repositorio incluye además un segundo conjunto en `Content/Species/SierraDemanda/` (haya, rebollo y pino silvestre) que no está en la lista; para usarlo basta con sustituir las entradas desde *Project Settings*. Conviene no mezclar ambos conjuntos sin recalibrar: los óptimos de nicho de cada uno se ajustaron para su propio escenario.

4.5. Puesta en marcha y comprobación

El nivel por defecto es `/Game/My_Levels/TestingMap`, tanto al abrir el editor como al ejecutar. La simulación arranca **en pausa** (`bStartPaused=True`), de modo que hay que sembrar una población y quitar la pausa desde la consola del editor (tecla `~` o `o` durante la ejecución):

```
# Sembrar 2000 plántulas y arrancar
Eco.SeedForest 2000
Eco.TogglePause

# 0 avanzar de forma controlada, 10 ticks (10 años simulados)
Eco.Step 10

# Diagnóstico del modelo
Eco.Demografia           # reparto por especie, estado y edad
```

```

Eco.Especies.Volcado      # rasgos declarados y sus derivados
Eco.PercentilesCampos     # percentiles de agua y nutrientes
Eco.Nicho.Mapa            # qué especie gana en cada celda
Eco.Luz.Perfil 0 0        # perfil vertical de luz en una columna

# Visualización
Eco.PaintWater / Eco.PaintNutrients / Eco.PaintLight / Eco.PaintVigor
Eco.Debug.Population 1
Eco.GrowHeroTree          # genera un árbol con geometría completa

# Rendimiento
Eco.Profile               # coste del tick por etapas
Eco.Frame.HUD 1           # presupuesto de fotograma en pantalla
Eco.Frame.Capture 30 prueba # captura 30 s a Saved/EcoProfile/prueba.csv
Eco.LOD.Stats             # reparto por niveles de detalle
stat EcoSim / stat EcoRender / stat Unit

```

El módulo registra **42 comandos** y **17 variables** de consola, todos con el prefijo `Eco.`; se pueden listar escribiendo ese prefijo en la consola. Además, las etapas del *tick* y del re-nivelado de detalle están instrumentadas como ámbitos de *Unreal Insights* y como categoría del perfilador CSV, de modo que una captura con `-trace=cpu,frame,bookmark,counters` muestra el reparto real del tiempo dentro del fotograma.

4.6. Ejecución de las pruebas automáticas

Las 31 pruebas se ejecutan desde el editor en *Tools* → *Session Frontend* → *Automation*, filtrando por `Eco.`. Cubren el determinismo del generador aleatorio y del relieve, la estabilidad de la erosión, el criterio de Nyquist en el ruido, las fórmulas de vigor y mortalidad, la compactación de la población, la estratificación vertical de la luz, la geometría del árbol (perfil de tronco, ángulo de inserción, reparto de ramas, costura de secciones, deformación del tronco) y el cálculo de tramos e histéresis del nivel de detalle. Ninguna necesita un mundo cargado, porque se apoyan en las partes puras del sistema.

También se pueden lanzar sin abrir la interfaz del editor:

```

UnrealEditor-Cmd.exe "<ruta>\ProceduralEcosystem.uproject" -ExecCmds="Automation RunTests
Eco.;Quit" -unattended -nopause -testexit="Automation Test Queue Empty"

```

4.7. Problemas conocidos y su solución

Síntoma	Causa y solución
El editor no abre los niveles o los materiales aparecen en blanco; los <code>.uasset</code> pesan unos pocos centenares de bytes.	Git LFS no estaba instalado al clonar y la copia de trabajo contiene punteros de texto. Solución: <code>git lfs install</code> y después <code>git lfs pull</code> .
Errores de compilación relacionados con rutas de inclusión o de biblioteca demasiado largas.	Comprobar que <code>Directory.Build.targets</code> sigue en la raíz del proyecto y acortar las rutas de instalación del motor y del repositorio.
El proyecto pide convertirse al abrirlo.	La versión de motor instalada no es la 5.7. Instalar la 5.7 o asociar el proyecto a ella con el botón derecho sobre el <code>.uproject</code> → <i>Switch Unreal Engine version</i> .
La simulación parece congelada nada más entrar en modo de juego.	Es el comportamiento previsto: arranca en pausa. Ejecutar <code>Eco.TogglePause</code> o <code>Eco.Step N</code> .
Hay árboles pero no se mueven, o no aparecen tocones al morir.	Faltan por asignar los recursos opcionales del apartado 4.4 (WindMPC, SnagMesh y demás).

Síntoma	Causa y solución
Dos corridas con la misma semilla dan resultados distintos.	Revisar que no se hayan modificado los parámetros de relieve y que el multiplicador de CO ₂ esté en el mismo estado en ambas: <code>bEnableC02Factor</code> cambia el resultado numérico. La comprobación rápida es <code>Eco.Fingerprint</code> , que imprime una huella del estado.
Caídas de rendimiento al aparecer muchos árboles nuevos.	Es el horneado de arquetipos. Se puede prehornear al arrancar con <code>bPrebakeLibraryOnStart=True</code> o forzarlo con <code>Eco.LOD.BakeAll</code> .

5. Estructura del proyecto

5.1. Vista general del repositorio

ProceduralEcosystem/	
├─ ProceduralEcosystem.uproject	Descriptor del proyecto: versión de motor, módulos y plugins
├─ Directory.Build.targets	Ajuste de MSBuild para rutas de inclusión largas
├─ .vsconfig	Componentes de Visual Studio requeridos
├─ .gitattributes	Reglas de Git LFS y normalización de fin de línea
├─ .gitignore	Artefactos generados que no se versionan
├─ README.md	
├─ Config/	CONFIGURACIÓN del proyecto
│ └─ DefaultEngine.ini	Motor: mapa por defecto, render (Lumen, Nanite, VSM), plataformas
│ └─ DefaultGame.ini	Calibración completa del ecosistema + lista de especies
│ └─ DefaultInput.ini	Entrada (configuración de plantilla, sin uso propio)
│ └─ DefaultEditor.ini	Vacío; reservado para ajustes del editor
├─ Content/	RECURSOS del proyecto (binarios, vía Git LFS)
│ └─ My_Levels/	
│ └─ TestingMap.umap	Nivel de pruebas; es el mapa por defecto
│ └─ Species/	Recursos USpeciesData, agrupados por escenario
│ └─ Garajonay/	Laurisilva: brezo arbóreo, laurel y til (activas)
│ └─ SierraDemanda/	Bosque templado: haya, rebollo y pino silvestre
│ └─ Materials/	
│ └─ M_EcoHeatmapDecal	Decal con el que se proyectan los mapas de calor
│ └─ Materials_TreeBark/	Corteza: texturas importadas y materiales derivados
│ └─ Textures/TreeBark_01..09	basecolor, normal y ORMH por juego
│ └─ Materials/Base/	Material maestro
│ └─ Materials/Instances/	Nueve instancias del maestro
│ └─ Meshes/	Mallas auxiliares de previsualización
├─ Source/	CÓDIGO PROPIO
│ └─ ProceduralEcosystem.Target.cs	Objetivo de compilación del juego
│ └─ ProceduralEcosystemEditor.Target.cs	Objetivo de compilación del editor
│ └─ ProceduralEcosystem/	
│ └─ ProceduralEcosystem.Build.cs	Reglas del módulo y dependencias
│ └─ ProceduralEcosystem.h/.cpp	Punto de entrada del módulo
│ └─ Public/	Cabeceras (interfaz del módulo)
│ └─ Private/	Implementación
└─ Docs/AnexoTecnico/	DOCUMENTACIÓN (este anexo y sus fuentes)
│ └─ anexo-tecnico.html	Fuente del documento
│ └─ anexo-tecnico.pdf	Documento entregable
│ └─ diagramas/	Diagramas UML en SVG y sus fuentes PlantUML

5.2. Separación de código propio, terceros, recursos y configuración

La estructura de un proyecto de Unreal Engine viene impuesta en buena medida por la herramienta: el motor espera encontrar el descriptor `.uproject` en la raíz, la configuración en `Config/`, los recursos en `Content/` y el código en `Source/`. Esa convención coincide con la separación exigida, de modo que el proyecto la cumple sin necesidad de forzar nada:

Elemento	Ubicación	Observaciones
Código propio	Source/	Todo el código escrito para el proyecto, sin excepción. Ningún fichero de código propio vive fuera de esta carpeta.

Elemento	Ubicación	Observaciones
Código de terceros	—	No hay ninguno dentro del repositorio. El motor y su plugin son software externo que se instala aparte, se referencia por versión desde <code>.upproject</code> y <code>Build.cs</code> , y por tanto no se copia en el árbol del proyecto. Esta es la práctica habitual y recomendada en Unreal Engine: incluir una copia del motor en el repositorio multiplicaría por decenas de gigabytes su tamaño sin ninguna ventaja.
Recursos	Content/	Separados por naturaleza: niveles, datos de especie, materiales de utilidad y materiales de corteza con sus texturas.
Configuración	Config/	Ficheros <code>.ini</code> del motor y del proyecto. La configuración de <i>compilación</i> es de otra naturaleza y vive junto al código, en los ficheros <code>.Build.cs</code> y <code>.Target.cs</code> , tal y como exige <i>UnrealBuildTool</i> .
Documentación	Docs/	Documento fuente, PDF generado y fuentes de los diagramas, de modo que estos se pueden regenerar y revisar en el control de versiones.

ÚNICO PUNTO EN EL QUE LA SEPARACIÓN NO ES TOTAL

Dentro de `Content/` conviven recursos creados para el proyecto (materiales, niveles y datos de especie) con recursos importados (las texturas de corteza). Están en carpetas distintas, pero no en árboles distintos, y el motivo es que el editor de Unreal Engine solo indexa lo que cuelga de `Content/`: una carpeta hermana quedaría fuera del sistema de recursos. La separación se logra, dentro de esa restricción, agrupando todo el material importado bajo `Content/Materials_TreeBark/Textures/`, distinguido además por el prefijo `T_` que el convenio de nomenclatura de Unreal reserva a las texturas.

5.3. Organización del código fuente

El módulo aplica la separación de Unreal Engine entre `Public/` —cabeceras visibles desde otros módulos— y `Private/` —implementación—. Dentro de cada una, la misma estructura de carpetas, una por componente del apartado 1: la ruta de un fichero indica a qué componente pertenece, y una cabecera y su implementación están siempre en la misma subcarpeta de una y otra rama.

Carpeta	.h / .cpp	Contenido
Core	4 / 1	<code>EcoCore.h</code> (generador aleatorio y flujos), <code>GridMath.h</code> (rejillas, CSR y Beer-Lambert), <code>EcoGeometry.h</code> (perpendicular estable) y <code>EcoStats.h/.cpp</code> (grupos de estadísticas e instrumentación).
Config	1 / 1	<code>EcosystemSettings.h/.cpp</code> : los 135 parámetros globales. La implementación está casi vacía a propósito, porque los valores por defecto viven en la cabecera y el resto lo aporta el <code>.ini</code> .
Species	1 / 1	<code>SpeciesData.h/.cpp</code> : el recurso de especie y sus enumerados.
Terrain	7 / 7	Campo 2D, relieve, ruido, erosión, agua, nutrientes y rejilla de luz gruesa.
Ecology	8 / 5	Reglas, vigor, modelo de carbono, población, <i>hash</i> espacial, depósitos de recurso, espacio de trabajo por <i>tick</i> y evento de muerte. Tres de las cabeceras son solo definiciones de estructura y por eso no tienen implementación asociada.
Simulation	1 / 1	<code>EcosystemSubsystem.h/.cpp</code> : el orquestador. Es el fichero más extenso del proyecto, porque además del bucle contiene los comandos de consola de diagnóstico.
Geometry	10 / 9	Colonización del espacio, esqueleto, nube de atractores, rejilla fina de luz, deformación de tronco, mallado, follaje, datos de viento y el actor del árbol <i>hero</i> .
Render	6 / 5	Gestor de nivel de detalle, biblioteca de arquetipos, horneado de mallas, actor anfitrión de instancias, clave de arquetipo y capa de suelo.

Carpeta	.h / .cpp	Contenido
Debug	3 / 4	Visualizador de campos, perfilador de fotograma, auditoría de especies y exportación de mapa de alturas. Esta última no tiene cabecera porque su única superficie pública es un comando de consola.
Test	0 / 1	EcoTests.cpp: las 31 pruebas de automatización. No expone cabecera porque nada depende de él; se compila condicionalmente con WITH_DEV_AUTOMATION_TESTS, de modo que no entra en una compilación de distribución.

5.4. Recursos (Content/)

- My_Levels/TestingMap.umap — nivel de trabajo, fijado como mapa por defecto del editor y del juego en DefaultEngine.ini.
- Species/ — un recurso USpeciesData por especie, con el prefijo SA_, agrupados por escenario ecológico en dos subcarpetas. Esta agrupación es lo que permite cambiar de bosque sustituyendo tres entradas de configuración.
- Materials/M_EcoHeatmapDecal — material de *decal* sobre el que la simulación proyecta la textura del visualizador de campos. Está separado de los materiales de árbol porque es una herramienta de diagnóstico, no parte del aspecto del bosque.
- Materials_TreeBark/ — texturas de corteza importadas, un material maestro y nueve instancias derivadas de él, además de las mallas auxiliares con las que se previsualizan. Usar instancias en lugar de nueve materiales independientes reduce las variantes de sombreador que hay que compilar.

5.5. Configuración

DefaultEngine.ini reúne los ajustes del motor: mapa por defecto, canal de render (Lumen con trazado por hardware, Nanite, mapas de sombras virtuales, texturas virtuales), formatos de sombreado por plataforma (DirectX 12 con *Shader Model 6* en Windows, Vulkan SM6 en Linux y Metal SM6 en macOS) y las redirecciones de nombre de la plantilla original del proyecto.

DefaultGame.ini contiene la sección [/Script/ProceduralEcosystem.EcosystemSettings], que es la calibración completa del ecosistema: está comentada por bloques temáticos, incluye las razones de los valores no evidentes y advierte de las claves que no deben tocarse. En la práctica funciona como el registro reproducible de la corrida, y por eso se versiona con el mismo cuidado que el código.

5.6. Artefactos generados y control de versiones

Todo lo que el motor y el compilador pueden regenerar queda fuera del repositorio mediante .gitignore: Binaries/, Build/, Intermediate/, DerivedDataCache/, Saved/, la solución de Visual Studio y los ficheros de usuario de los distintos entornos de desarrollo. El fichero deja constancia expresa de que Content/, Config/ y Source/ sí se versionan, para que la ausencia de una regla no se confunda con un olvido.

.gitattributes cumple dos funciones. Encamina hacia Git LFS todos los formatos binarios —recursos y niveles de Unreal, imágenes, mapas de altura en bruto, modelos, audio y tipografías—, lo que mantiene el historial de Git manejable. Y normaliza a LF los finales de línea de .h, .cpp y .cs, de modo que el código no genera diferencias espurias al trabajar desde sistemas distintos.

5.7. Observaciones sobre la estructura

Dos detalles que conviene dejar registrados por honestidad documental, ambos sin efecto sobre el funcionamiento:

- `Source/ProceduralEcosystem/Public/Geometry/GeometryMath.h` define el espacio de nombres `EcoGeom` con una función que quedó sustituida por `EcoGeometry::AnyPerpendicular`, en `Core/EcoGeometry.h`, al unificar tres copias del mismo cálculo. Ningún fichero lo incluye y ningún símbolo suyo se usa: es un residuo de esa refactorización y puede eliminarse.
- Los comentarios de `DefaultGame.ini` remiten a un documento `Docs/Fase6_Guia.md` con las instrucciones para crear los recursos opcionales (*Material Parameter Collections* de viento y estación, mallas y materiales de la capa de suelo). Ese documento no está en el repositorio; el apartado 4.4 de este anexo recoge, mientras tanto, qué recurso corresponde a cada clave y qué ocurre si no se asigna.